

FIRST RUN TUTORIAL

Unitree L1 First Run Tutorial

UniLiDAR to RViz2
Stationary OctoMap in Docker

First edition
Prepared 29 July 2026
Validated project baseline 07616c2
Audience: First-time operators and ROS 2 project users

DOCUMENT CONTROL

This document is generated from `first-run-tutorial.md`.
Commands and paths describe the repository state identified above.

Operate the powered LiDAR only after reading the safety section.

This tutorial takes a new operator through one complete, safe first run. It starts with the project build, detects the Unitree L1 without changing the host, proves that live UniLiDAR data is arriving, opens the raw point cloud in RViz2, creates a stationary OctoMap, saves it, and reopens the saved map.

Every command is entered on the host from the repository root. The project scripts put ROS 2 Humble, the Unitree driver, RViz2, and OctoMap inside the project container. Do not open a host-native ROS terminal for this procedure.

The result you are working toward

Stage	Evidence of success	What it proves
Software	SMOKE_TEST_PASS	The image, workspace, driver, and launch files are ready
GUI	GUI_SMOKE_TEST_PASS	X11 and hardware rendering reach the container
UniLiDAR	LIDAR_DATA_VALIDATION_PASS	Real cloud, IMU, rates, and diagnostics are present
RViz2	A stable coloured point cloud	The raw L1 stream can be interpreted visually
OctoMap	OCTOMAP_MAPPING_HEALTH_PASS	Non-empty occupied markers and binary map data exist
Saved map	OCTOMAP_INSPECT_PASS	The map file has a valid header and non-zero nodes

ENGINEERING NOTE

A live driver process is not proof that the L1 is working. This tutorial treats actual messages, measured rates, diagnostics, and non-empty map data as the success conditions.

Terminal names used in this tutorial

Three terminal windows make the sequence easy to follow.

- **Terminal A** owns the live L1 runtime and later the raw RViz2 window.
- **Terminal B** validates the runtime and later owns live OctoMap.
- **Terminal C** evaluates, saves, and inspects the map.

Commands such as `START_RVIZ=true` apply only to the command on that line. Exported values such as `LIDAR_DEVICE` remain in the terminal where they were set. Use lowercase `true` and `false`; the scripts reject other Boolean forms.

What the project starts

The live data path is deliberately small:

```

L1 and Unitree adapter
|
v
/dev/unitree_lidar in Docker
|
v
unitree_lidar_ros2_node
|
+--> /unilidar/cloud --> RViz2 --> octomap_server
|
+--> /unilidar/imu
|
+--> ll_monitor --> /diagnostics

```

For the stationary mapping exercise, the project also publishes an identity transform from `map` to `unilidar_lidar`. That transform is correct only while the L1 housing remains fixed.

1. Prepare the Computer

1.1 Requirements

Complete the software preparation before connecting or powering the L1.

Requirement	Why it is needed
Docker Engine	Runs the pinned Ubuntu 22.04 and ROS 2 Humble environment
Docker Compose	Adds only the requested GUI and serial-device access
Internet on first build	Downloads the base image and pinned dependencies
X11 graphical session	Displays RViz2 from inside Docker
<code>/dev/dri</code> devices	Provides direct OpenGL rendering to RViz2
Unitree adapter	Converts the L1 connection to the supported USB serial path
Separate 12 V / 1 A supply	Powers the L1; USB power is not sufficient

The Ubuntu host is the Docker client and X11 display server, not the ROS runtime. The container supplies Jammy, ROS 2 Humble, RViz2, the Unitree driver, OctoMap, and all build dependencies.

SAFETY / WARNING

Do not install ROS 2 Humble on the Ubuntu 24.04 host for this project. Do not add `privileged: true`, use `chmod 777`, or grant global X access with `xhost +`.

1.2 Open the repository

Open a normal host terminal and change to the cloned repository. Substitute the actual clone location if it differs:

```
cd /path/to/Unitree-L1-ROS2-Octomap
```

Confirm the documented project baseline:

```
git rev-parse --short HEAD
```

The tutorial was prepared against:

```
07616c2
```

A later compatible commit can also be used, but commands and expected behavior should be checked against that revision's documentation.

1.3 Build the image and workspace once

Run the three commands in order:

```
./scripts/docker-build.sh
./scripts/workspace-build.sh
./scripts/smoke-test.sh
```

The image build creates `unitree-l1:humble-v1.0.16`. The workspace build retrieves and verifies the pinned Unitree and OctoMap source checkouts, installs declared dependencies, and builds six ROS 2 packages.

The generated colcon directories are:

```
ros2_ws/build/
ros2_ws/install/
ros2_ws/log/
```

They belong at the workspace root. No `build`, `install`, or `log` directory may appear below `ros2_ws/src`.

EXPECTED RESULT

The final command prints `SMOKE_TEST_PASS`. It also proves that ROS 2 Humble, colcon, RViz2, the Unitree driver executable, the monitor, and the project launch files are installed inside Docker.

If the first build is interrupted, correct the reported Docker, network, or dependency problem and run the same commands again. Do not copy installed files into `ros2_ws/src`, and do not build from a host ROS installation.

1.4 Understand the pinned inputs

The first build recreates two ignored third-party checkouts:

Dependency	Project location	Pinned revision
Unitree UniLiDAR SDK	<code>ros2_ws/src/unilidar_sdk/</code>	<code>1bd7d95d8ab7ce7a22058d2bb07e39fd62612aa6</code>
OctoMap mapping	<code>ros2_ws/src/octomap_mapping/</code>	<code>f79da9a9a1fcdf82e72dab4df288d6cc27c6e163</code>

Project-specific behavior remains in the three tracked `l1_*` packages. The dependency fetcher rejects unexpected revisions and local modifications, so a first run cannot silently use a different vendor implementation.

2. Prove the GUI Path

RViz2 will run in Docker and display through the current host X11 session. Prove this path before hardware work so a graphical problem cannot be confused with a sensor problem.

2.1 Check the graphical session

Run these commands in the same graphical terminal that will start RViz2:

```
printf 'DISPLAY=%s\n' "${DISPLAY:-<unset>}"
printf 'XAUTHORITY=%s\n' "${XAUTHORITY:-<unset>}"
test -n "${DISPLAY:-}"
test -n "${XAUTHORITY:-}"
test -r "$XAUTHORITY"
test -c /dev/dri/card0
test -c /dev/dri/renderD128
```

The two variables must identify the active graphical session. `XAUTHORITY` must name a readable cookie file. The current project wrappers explicitly require `/dev/dri/card0` and `/dev/dri/renderD128`.

ENGINEERING NOTE

Do not invent an Xauthority path and do not replace cookie authentication with `xhost +`. If a variable is missing, open a terminal from the active graphical desktop and use that session's real values.

2.2 Run the graphical smoke test

```
./scripts/gui-smoke-test.sh
```

The script builds a temporary GUI-enabled container view, verifies the Docker runtime boundary, checks X11 with `xdpyinfo`, displays OpenGL renderer information with `glxinfo`, and confirms that RViz2 can start.

The final line has this form:

```
GUI_SMOKE_TEST_PASS display=<active-display>
```

EXPECTED RESULT

The renderer information identifies working DRI/OpenGL, and the command ends with `GUI_SMOKE_TEST_PASS`.

2.3 Why this test comes first

The GUI overlay adds three things when the runtime container is created:

- the read-only host X11 socket;
- the read-only Xauthority cookie;
- the host DRI devices and their group IDs.

Those mounts and devices cannot be added to an already running headless container. This fact matters later: the runtime used for an OctoMap RViz2 window must originally be started with `START_RVIZ=true`.

3. Connect and Detect the L1**3.1 Read the safety boundary**

The L1 contains a moving scanning mechanism and uses a separate power supply. Treat wiring and operation as hardware work, even though the software procedure is read-only.

SAFETY / WARNING

Secure the L1 housing, clear the mechanism's operating area, and disconnect power before changing any wiring.

Use the Unitree cable and adapter specified for the L1. Observe the documented 12 V / 1 A requirement and polarity. USB connects the adapter to the computer; it is not the L1 power supply.

- Do not power the L1 from 5 V USB.
- Do not connect 3.3 V TTL directly to USB or RS-232.
- Keep hands, clothing, loose cables, and objects clear.
- Do not change the 2,000,000 baud driver setting without evidence.
- Do not move or rewire powered equipment during the tutorial.

3.2 Record the host state before connection

With the L1 disconnected, run:

```
./scripts/check-lidar.sh
```

This gives a useful before-state. The script lists USB devices, stable serial links, resolved `ttyUSB*` or `ttyACM*` devices, ownership, numeric group, USB identity properties, and any process that currently has a candidate port open.

It ends with:

```
No permissions, services, udev rules or network settings were changed.
```

3.3 Connect and power

With power disconnected, complete the physical wiring:

1. Secure the L1 in its operating position.
2. Connect the L1 to the Unitree adapter.
3. Connect the adapter to the PC USB port.
4. Check the separate supply voltage and polarity.
5. Clear the mechanism and then apply L1 power.

The mechanism may begin moving. Do not touch it to test whether it is active.

3.4 Identify the new serial device

Run the same read-only check again:

```
./scripts/check-lidar.sh
```

Look for a newly reported stable link and its resolved character device. A typical shape is:

```
link=/dev/serial/by-id/<adapter-identity>
resolved=/dev/ttyUSB<number>
open_by_process=none_detected
```

Actual identity, tty number, VID, PID, and serial values are host-specific. Record what the script reports rather than copying example values.

EXPECTED RESULT

One identified adapter resolves to a `ttyUSB*` or `ttyACM*` character device and reports `open_by_process=none_detected`.

3.5 Select the device when detection is ambiguous

If exactly one stable `/dev/serial/by-id` candidate exists, the launcher selects its resolved tty automatically. No export is needed.

If several adapters exist, set the stable adapter explicitly in Terminal A. Replace the placeholder with the exact link reported above:

```
export LIDAR_DEVICE="$(
  readlink -e /dev/serial/by-id/<identified-adapter>
)"
test -c "$LIDAR_DEVICE"
```

If the host provides no stable link, use the exact `resolved=` value reported by `check-lidar.sh`:

```
export LIDAR_DEVICE=/dev/ttyUSB0
test -c "$LIDAR_DEVICE"
```

The tty number above is only an example. Do not assume that reconnecting the adapter will preserve it.

The launch wrapper resolves the path again, requires a `ttyUSB*` or `ttyACM*` character device, checks whether another process has it open, and calculates the device GID. Do not set `LIDAR_GID` manually for the normal workflow.

4. Start UniLiDAR without RViz2

The first live run is headless. This separates serial and ROS data validation from all graphical concerns.

4.1 Start the driver in Terminal A

```
START_RVIZ=false ./scripts/lidar-launch.sh
```

Keep Terminal A open. The wrapper performs these checks before starting ROS:

1. Resolve exactly one selected serial character device.
2. Refuse unexpected device names and a busy port.
3. Refuse an existing `unitree_l1_runtime` container.
4. Calculate the host tty group ID.
5. Map the device to `/dev/unitree_lidar` in an unprivileged container.
6. Prove container read and write access.
7. Start the Unitree driver and the read-only L1 monitor.

Representative startup markers are:

```
LIDAR_CONTAINER_ACCESS_PASS
Runtime container=unitree_l1_runtime, monitor=true, rviz=false.
```

The exact host tty and numeric group ID vary by computer.

EXPECTED RESULT

Terminal A remains active, reports no serial-access error, and begins printing driver and monitor output.

The launch passes the container port to `unitree_lidar_ros2_node`. The project configuration requests an 18-scan cloud, publishes the cloud in `unilidar_lidar`, and publishes IMU data in `unilidar_imu`.

4.2 Require real data in Terminal B

Open Terminal B at the repository root:

```
./scripts/lidar-validate.sh
```

This command enters the named runtime container and checks:

- the expected nodes and topics;
- the configured driver port;
- exact cloud and IMU message types;

- one actual message from each stream;
- a measured average rate from each stream;
- one diagnostics message from `l1_monitor`.

The key success lines are:

```
MESSAGE_PASS topic=/unilidar/cloud type=sensor_msgs/msg/PointCloud2
MESSAGE_PASS topic=/unilidar/imu type=sensor_msgs/msg/Imu
RATE_PASS topic=/unilidar/cloud
RATE_PASS topic=/unilidar/imu
LIDAR_DATA_VALIDATION_PASS
```

The complete output is also written below the ignored `logs/tests/` directory.

EXPECTED RESULT

Both message checks, both rate checks, and the final `LIDAR_DATA_VALIDATION_PASS` marker appear.

4.3 Confirm that ROS remains in Docker

Still in Terminal B:

```
REQUIRE_RVIZ=false ./scripts/verify-docker-only.sh
```

The expected final marker is:

```
DOCKER_ONLY_PIPELINE_PASS container=unitree_l1_runtime rviz_required=false
```

4.4 Interpret the live interfaces

Interface	ROS type	Default frame or role
/unilidar/cloud	sensor_msgs/msg/PointCloud2	unilidar_lidar
/unilidar/imu	sensor_msgs/msg/Imu	unilidar_imu
/diagnostics	diagnostic_msgs/msg/DiagnosticArray	Stream health

The monitor expects at least 5 Hz cloud and 20 Hz IMU after startup. Prior real hardware validation observed about 8 to 10 Hz cloud and 210 to 250 Hz IMU. Measured values vary, so the script checks for a real average rather than requiring one exact rate.

The driver can remain alive even when initialization did not produce data. That is why node names or publishers alone do not count as success.

4.5 Stop the headless runtime

Press `Ctrl-C` once in Terminal A and allow Docker Compose to remove the temporary runtime. Then inspect:

```
docker ps --filter name=unitree_l1_runtime
```

Only the heading should remain. If a row is present, inspect it rather than starting another runtime or deleting it blindly.

The headless container must be stopped because the next step recreates it with the GUI socket, cookie, and DRI devices.

5. Open the Raw Cloud in RViz2

5.1 Start a GUI-enabled runtime in Terminal A

Return to Terminal A. If `LIDAR_DEVICE` was selected explicitly, keep using the same terminal so the export remains available.

```
START_RVIZ=true ./scripts/lidar-launch.sh
```

The wrapper repeats all device and access checks, adds the GUI overlay, starts the driver and monitor, and launches RViz2 in the same container.

Representative startup output includes:

```
LIDAR_CONTAINER_ACCESS_PASS
Runtime container=unitree_l1_runtime, monitor=true, rviz=true.
```

EXPECTED RESULT

An RViz2 window opens and a coloured point cloud updates continuously while Terminal A remains active.

5.2 Know what the raw RViz2 profile displays

Setting	Project value
Fixed Frame	unilidar_lidar
Display type	PointCloud2
Topic	/unilidar/cloud
Reliability	Reliable
Durability	Volatile
Colour	Height-axis colour
Point style	Three-pixel points

The Unitree driver does not publish a transform tree for raw visualization. Using `unilidar_lidar` as the fixed frame is therefore deliberate. The view shows measurements in the sensor frame; it does not yet show a world-aligned map. Use the mouse to orbit, pan, and zoom. A healthy first view has stable scene geometry, a continuous refresh, and no persistent PointCloud2 error in the Displays panel.

5.3 Recheck the runtime boundary

In Terminal B:

```
./scripts/verify-docker-only.sh
```

Required verdicts include:

```
HOST_NATIVE_RVIZ_ABSENT
DOCKER_ONLY_PIPELINE_PASS container=unitree_l1_runtime rviz_required=true
```

You may also repeat live validation:

```
./scripts/lidar-validate.sh
```

5.4 Leave this runtime running

Do not stop Terminal A before starting live OctoMap. The mapping wrapper uses `docker exec` to add OctoMap to this same ROS graph.

ENGINEERING NOTE

Keep the GUI-enabled runtime running. A runtime originally created with `START_RVIZ=false` cannot later gain the X11 socket, Xauthority mount, or DRI devices needed by `OCTOMAP_RVIZ=true`.

The OctoMap step deliberately opens a second RViz2 window with a different fixed frame and display profile. Minimize the raw window if screen space is limited, but keep the live runtime active.

6. Build a Stationary OctoMap

This first mapping exercise is a fixed-sensor bench test. OctoMap integrates range observations into occupied and free space, but it does not estimate the L1 pose.

6.1 Freeze the sensor pose

Place the secured L1 in the position from which the map will be made. Do not translate or rotate its housing after mapping begins. Normal internal scanner motion is expected; the external sensor pose must remain fixed.

SAFETY / WARNING

`STATIC_SENSOR=true` publishes an identity map to `unilidar_lidar` transform. Moving the housing while that transform is active creates geometrically false map data.

The stationary transform is:

```
map -> unilidar_lidar
translation: 0, 0, 0
rotation:    0, 0, 0
```

This declares the sensor origin to be the map origin for the bench exercise.

6.2 Start OctoMap in Terminal B

With Terminal A and raw RViz2 still running:

```
STATIC_SENSOR=true \
OCTOMAP_RVIZ=true \
./scripts/octomap-launch.sh
```

Terminal B now owns the OctoMap process and its RViz2 window. Representative startup output is:

```
OCTOMAP_LAUNCH_READY container=unitree_l1_runtime \
static_sensor=true rviz=true
Starting bench OctoMap; keep the L1 stationary and press Ctrl-C to stop.
```

EXPECTED RESULT

A second RViz2 window opens with Fixed Frame `map`. The live cloud appears and occupied OctoMap voxels begin to accumulate.

Two RViz2 windows are expected: the raw view uses fixed frame `unilidar_lidar`, while the map view uses `map` and shows both the current cloud and occupied voxels.

The second window is not a duplicate configuration. It uses the `map` frame and subscribes to `/occupied_cells_vis_array`.

6.3 Understand the mapping defaults

Parameter	Default	Meaning
Input cloud	<code>/unilidar/cloud</code>	PointCloud2 observations from the driver
Frames	<code>map / unilidar_lidar</code>	World accumulation / cloud frame
Resolution	0.10 m	OctoMap voxel edge length
Maximum range	15.0 m	Maximum sensor ray length used for mapping
Sensor model	hit 0.7, miss 0.4, clamp 0.12 to 0.97	Occupancy evidence and limits

Ground-plane filtering and speckle filtering are disabled. Free space is not published as a visualization layer. The RViz2 map profile therefore focuses on occupied voxels, not every internal free-space update.

6.4 What success looks like

The map view should show recognisable occupied surfaces around the fixed sensor. The exact density depends on scene geometry and observation time. The following ROS interfaces are the important outputs:

Interface	Purpose
<code>/occupied_cells_vis_array</code>	Transient-local marker array displayed by RViz2
<code>/octomap_binary</code> topic	Binary OctoMap message used by evaluation
<code>/octomap_binary</code> service	Map source used by the saver

Allow several scans to arrive before evaluating. The L1 cloud must remain live, and a valid `map` to `unilidar_lidar` transform must exist for the cloud time.

ENGINEERING NOTE

A visually plausible stationary map is not a mobile SLAM result. It contains no estimated trajectory and provides no evidence of localization accuracy.

7. Evaluate, Save, and Inspect

Keep Terminals A and B running throughout this section. Use Terminal C for checks and file creation.

7.1 Evaluate non-empty map output

In Terminal C:

```
./scripts/evaluate-octomap.sh
```

The evaluator requires the `/octomap_server` node, identifies the mapping mode, subscribes with reliable transient-local QoS, and waits for both occupied markers and a binary map.

Successful output has this form:

```
mapping_mode=stationary_bench_mapping
OCTOMAP_MAPPING_HEALTH_PASS markers=<count> \
occupied_markers=<count> occupied_points=<count> \
resolution_m=0.1 binary_payload_bytes=<count> \
map_id=OcTree frames=map
```

Counts depend on the scene. They must be non-zero.

EXPECTED RESULT

The mode is `stationary_bench_mapping`, and the final health marker reports at least one occupied marker, one occupied point, and a non-empty binary payload.

If evaluation is attempted too quickly, allow more live scans and repeat it. Do not move the L1 to force additional coverage while stationary mode is active.

7.2 Choose a safe map name

The save script accepts a safe basename ending in `.bt` or `.ot`. This tutorial uses:

```
first_room.bt
```

Names may contain letters, digits, dots, underscores, and hyphens. They cannot contain a directory separator or begin with punctuation. The script refuses to overwrite an existing file.

7.3 Save the live map

In Terminal C:

```
./scripts/save-octomap.sh first_room.bt
```

The wrapper creates the ignored project `maps/` directory when needed, asks the running OctoMap server to save into the mounted workspace, and requires a non-empty host file.

Successful output has this form:

```
Requesting OctoMap save to <project>/maps/first_room.bt
OCTOMAP_SAVE_PASS file=<project>/maps/first_room.bt bytes=<count>
```

EXPECTED RESULT

`OCTOMAP_SAVE_PASS` appears with a byte count greater than zero.

If the name already exists, select a new descriptive name rather than deleting the earlier result during the first-run workflow:

```
./scripts/save-octomap.sh first_room_02.bt
```

7.4 Inspect the saved file

```
./scripts/inspect-octomap.sh first_room.bt
```

Inspection does not start ROS. It checks the OctoMap header, tree identifier, stored-node count, resolution, data marker, size, modification time, and SHA-256 checksum.

Successful output has this form:

```
file=<project>/maps/first_room.bt
format=bt tree_id=OcTree stored_nodes=<count> resolution_m=0.1
bytes=<count> modified=<timestamp>
sha256=<digest>
OCTOMAP_INSPECT_PASS map=first_room.bt
```

EXPECTED RESULT

The stored-node count and byte count are non-zero, and the final line is `OCTOMAP_INSPECT_PASS map=first_room.bt`.

7.5 Understand what has been proven

At this point the workflow has proved:

- the selected serial adapter is accessible without broad host permissions;
- the pinned Unitree driver publishes real cloud and IMU messages;
- stream rates and diagnostic messages are present;
- RViz2 renders the raw cloud from inside Docker;
- OctoMap receives the cloud through a valid stationary transform;
- occupied markers and binary OctoMap data are non-empty;
- a saved map file is structurally valid and contains nodes.

It has not proved localization, mobile trajectory quality, loop closure, large-area completeness, or metric map accuracy. Those require a dynamic pose source and separate evaluation.

8. Shut Down and Reopen the Map

8.1 Shut down in reverse order

Save and inspect the map before stopping either live process.

1. Press `Ctrl-C` in Terminal B.
2. Wait for the OctoMap launch and second RViz2 window to close.
3. Press `Ctrl-C` in Terminal A.
4. Wait for the raw RViz2, monitor, driver, and runtime to close.
5. Inspect remaining project containers.

```
docker ps --filter name=unitree_l1
```

No live project runtime row should remain. The project uses `run --rm`, so a cleanly stopped temporary container is removed automatically.

SAFETY / WARNING

Do not unplug, reconnect, or change L1 wiring while power is applied. Stop software, remove power, wait for mechanical motion to cease, and only then change connections.

8.2 Reopen the saved map without hardware

Leave the L1 stopped. From a graphical terminal at the repository root:

```
./scripts/view-octomap.sh first_room.bt
```

The viewer first runs the same structural inspection. It then creates an independent container, uses ROS domain 43 by default, loads the saved map into `octomap_server`, and opens the saved-map RViz2 profile.

Representative success markers are:

```
OCTOMAP_INSPECT_PASS map=first_room.bt
VIEW_SAVED_OCTOMAP_READY map=first_room.bt domain=43 rviz=true
```

EXPECTED RESULT

RViz2 opens with Fixed Frame `map` and displays the saved occupied voxels without the L1 connected.

Press `Ctrl-C` to close the viewer and its map server.

8.3 Reconnect safely on a later run

After any USB disconnect or power cycle:

- secure and wire the L1 with power removed;
- rerun `./scripts/check-lidar.sh`;
- confirm the adapter identity and current resolved tty;
- recreate the live runtime with `./scripts/lidar-launch.sh`;
- repeat `./scripts/lidar-validate.sh`.

Never assume `/dev/ttyUSB0` still denotes the same adapter after reconnection.

8.4 Runtime data stays out of Git

The tutorial can create these local, ignored paths:

Path	Content
<code>ros2_ws/build/</code>	Colcon package build output
<code>ros2_ws/install/</code>	Installed workspace overlay
<code>ros2_ws/log/</code>	Colcon logs
<code>logs/tests/</code>	Timestamped live validation output
<code>maps/</code>	Saved <code>.bt</code> and <code>.ot</code> maps

These files are operational results, not project source. Do not add them to Git or copy them into `ros2_ws/src`.

9. Quick Command Card

Use this one-page card only after reading the safety and explanation sections.

```
# Build once and prove the GUI.
cd /path/to/Unitree-L1-ROS2-Octomap
./scripts/docker-build.sh
./scripts/workspace-build.sh
./scripts/smoke-test.sh
./scripts/gui-smoke-test.sh

# Run before and after safe connection and power-up.
./scripts/check-lidar.sh

# Use only when more than one stable adapter exists.
export LIDAR_DEVICE="$(
  readlink -e /dev/serial/by-id/<identified-adapter>
)"
test -c "$LIDAR_DEVICE"

# Terminal A: first prove the driver headlessly.
START_RVIZ=false ./scripts/lidar-launch.sh

# Terminal B: require real data and the Docker boundary.
./scripts/lidar-validate.sh
REQUIRE_RVIZ=false ./scripts/verify-docker-only.sh

# Press Ctrl-C in Terminal A, then recreate it with GUI access.
START_RVIZ=true ./scripts/lidar-launch.sh

# Terminal B: verify raw RViz2, then leave Terminal A running.
./scripts/verify-docker-only.sh

# Terminal B; keep the L1 housing fixed.
STATIC_SENSOR=true \
OCTOMAP_RVIZ=true \
./scripts/octomap-launch.sh

# Terminal C: prove, save, and inspect non-empty map data.
./scripts/evaluate-octomap.sh
./scripts/save-octomap.sh first_room.bt
./scripts/inspect-octomap.sh first_room.bt

# Press Ctrl-C in Terminal B, then in Terminal A.
docker ps --filter name=unitree_l1

# Reopen without hardware; press Ctrl-C to close the viewer.
./scripts/view-octomap.sh first_room.bt
```

EXPECTED RESULT

Required markers are SMOKE_TEST_PASS, GUI_SMOKE_TEST_PASS, LIDAR_DATA_VALIDATION_PASS, OCTOMAP_MAPPING_HEALTH_PASS, OCTOMAP_SAVE_PASS, and OCTOMAP_INSPECT_PASS.

SAFETY / WARNING

Remove L1 power and wait for motion to stop before disconnecting or changing hardware.

10. Troubleshooting

Work from the first failed stage. Do not mask one problem by broadening permissions or adding host software.

10.1 Fast diagnosis table

Symptom	Likely boundary	Safe next action
Docker build fails	Image, network, or dependency	Read the first error, correct it, and rerun the build wrapper
No serial candidate	USB, cable, adapter, or power	Compare before and after <code>check-lidar.sh</code> output
Several serial candidates	Device selection	Export the resolved identity for the intended adapter
Device already open	Port ownership	Use the reported process information; stop nothing by assumption
Node alive but no messages	Power, L1 state, or serial stream	Require <code>lidar-validate.sh</code> ; a node name is not success
GUI smoke test fails	X11 cookie or DRI	Return to the active graphical session and test real values
Raw RViz2 is blank	Cloud stream, frame, or display status	Repeat live validation and inspect the PointCloud2 display
OctoMap RViz2 fails to open	Runtime was created headlessly	Stop it and recreate with <code>START_RVIZ=true</code>
No occupied voxels	Cloud, TF, range, or insufficient scans	Keep the sensor fixed, verify live data, and allow more scans
Save refuses a name	Existing or unsafe basename	Choose a new safe <code>.bt</code> or <code>.ot</code> basename
Saved viewer fails	File, GUI, or stale viewer name	Inspect the map, run the GUI smoke test, and inspect containers

10.2 Serial device is absent

Run:

```
./scripts/check-lidar.sh
```

Compare the disconnected and connected outputs. Check the Unitree adapter, USB cable, USB port, separate L1 supply, and physical connector seating with power removed.

The runtime launcher intentionally auto-detects only stable `/dev/serial/by-id` entries. If the host exposes a valid tty without a stable link, select the exact `resolved=` device using `LIDAR_DEVICE`.

Do not create a permissive udev rule merely to make the error disappear. The launcher passes the actual tty group ID into the unprivileged container.

10.3 Serial device is busy

Both `check-lidar.sh` and `lidar-launch.sh` use `fuser` when available. If the port is open, the launcher refuses to continue and stops no process.

Identify why the reported process owns this specific port. Close a previous project run cleanly if it is the owner.

ModemManager should not be disabled pre-emptively. A targeted ignore rule should be considered only after repeated evidence identifies the adapter by VID, PID, and serial number.

10.4 Driver exists but validation fails

Repeat:

```
./scripts/lidar-validate.sh
```

Use the first missing marker to narrow the problem:

- No cloud message suggests that the serial stream is not producing points.
- No IMU message suggests incomplete live output.
- No average rate means messages did not continue during the sample window.
- No diagnostics message means the monitor is absent or cannot observe data.

Stop Terminal A with `Ctrl-C` before changing power or wiring. Check the selected device and L1 operating state. The pinned vendor ROS 2 node does not make process liveness a reliable initialization verdict.

Do not silently modify the ignored vendor checkout. Any necessary adaptation belongs in tracked project code and must check initialization results, be tested, and be documented.

10.5 RViz2 does not open

Run the preflight again:

```
printf 'DISPLAY=%s\n' "${DISPLAY:-<unset>}"
printf 'XAUTHORITY=%s\n' "${XAUTHORITY:-<unset>}"
test -r "$XAUTHORITY"
test -c /dev/dri/card0
test -c /dev/dri/renderD128
./scripts/gui-smoke-test.sh
```

Use the active desktop's X11 cookie. Do not grant global access. The Compose overlay mounts the socket and cookie read-only and adds only DRI device access.

If raw RViz2 opens but has no points, check its Fixed Frame is `unilidar_lidar`, its topic is `/unilidar/cloud`, and the PointCloud2 display is enabled. Then repeat `lidar-validate.sh`.

10.6 OctoMap RViz2 fails after a headless launch

The symptom commonly occurs when Terminal A was started with:

```
START_RVIZ=false ./scripts/lidar-launch.sh
```

`docker exec` can add an environment variable, but it cannot add the missing X11 and DRI mounts to that running container. Stop Terminal A cleanly, confirm the runtime is gone, and recreate it:

```
START_RVIZ=true ./scripts/lidar-launch.sh
```

Leave it running, then retry in Terminal B:

```
STATIC_SENSOR=true \
OCTOMAP_RVIZ=true \
./scripts/octomap-launch.sh
```

10.7 OctoMap remains empty

First confirm the cloud is still healthy:

```
./scripts/lidar-validate.sh
```

Then evaluate:

```
./scripts/evaluate-octomap.sh
```

Check these conditions:

- the L1 housing has remained physically fixed;
- the raw cloud contains surfaces within the 15 m mapping range;
- `/octomap_server` is running only once;
- the stationary transform is active;
- enough scans have arrived to form occupied cells.

The default map profile does not display free space. Ground and speckle filters are disabled, so this first result is an occupancy demonstration rather than a finished navigation map.

10.8 OctoMap reports that it is already running

Only one `/octomap_server` is allowed in the live graph by the wrapper. Return to the earlier Terminal B and stop it with `Ctrl-C`. Confirm that launch has ended before trying again. Do not start repeated background instances.

10.9 Map save or inspection fails

Save while the live OctoMap process in Terminal B is still running:

```
./scripts/save-octomap.sh first_room_02.bt
```

The saver requires the `/octomap_binary` service and a new safe filename. Inspection requires a readable file within the `project maps/` directory:

```
./scripts/inspect-octomap.sh first_room_02.bt
```

An empty file, invalid header, zero stored nodes, or missing data marker is a real failure. Do not rename arbitrary data to `.bt` to bypass inspection.

10.10 A runtime container name already exists

Inspect before acting:

```
docker ps -a --filter name=unitree_l1_runtime
```

If it is running, find the terminal or process that owns the earlier workflow and stop it cleanly. If it is stopped, determine why automatic removal did not complete before deciding how to handle that specific container. Do not remove unrelated containers.

10.11 Moving-platform mapping

This tutorial intentionally uses a stationary sensor. On a moving robot:

```
STATIC_SENSOR=false \  
OCTOMAP_RVIZ=true \  
./scripts/octomap-launch.sh
```

That command is valid only when an external pose system already publishes a correct, time-varying transform from `map` to `unilidar_lidar`.

SAFETY / WARNING

This repository does not currently provide the required odometry or SLAM pose source. Do not move the robot with the stationary identity transform enabled.

Mobile validation must separately evaluate trajectory accuracy, transform timing, LiDAR-to-IMU extrinsics, map consistency, and failure behavior. OctoMap creates occupancy from posed measurements; it cannot replace localization.

10.12 Final success checklist

The first run is complete when all of the following are true:

- `SMOKE_TEST_PASS` was obtained after the workspace build.
- `GUI_SMOKE_TEST_PASS` was obtained from the graphical session.
- The adapter identity and resolved tty were recorded.
- `LIDAR_DATA_VALIDATION_PASS` proved real cloud and IMU streams.
- Raw RViz2 displayed a continuously updating point cloud.
- The L1 housing remained fixed throughout stationary mapping.
- `OCTOMAP_MAPPING_HEALTH_PASS` reported non-empty map output.
- `OCTOMAP_SAVE_PASS` created a new map in `maps/`.
- `OCTOMAP_INSPECT_PASS` reported non-zero stored nodes.
- The saved map reopened in the isolated viewer.
- Live containers stopped cleanly.
- L1 power was removed before any hardware disconnection.

Retain the saved map and validation log locally for comparison with later runs. They are intentionally ignored by Git.